ESPCI PARIS | PSL

Résolution numérique de problèmes de physique quantique

Maxime Rimbaud

144

Table des matières

1 L'oscillateur harmonique dans tous ses états (quantiques) 3
1.1 Étude préliminaire . . . 3
1.1.1 Puits de potentiel infini . . . 3
1.1.2 Potentiel harmonique dans un puits infini . . . 4
1.2 Potentiel normalisé et solutions . . . 4
1.3 Résolution numérique . . . 5
1.3.1 Symétries de la matrice **H** . . . 5
1.3.2 Energies propres . . . 7
1.3.3 Fonctions propres . . . 9

2 Puits de potentiel fini dans les hétérostructures de semi-conducteurs 11
2.1 Modèle du puits de potentiel . . . 11
2.2 Résolution numérique . . . 12
2.2.1 Base de fonctions propres . . . 12
2.2.2 Solutions numériques . . . 12
2.2.3 Lien avec les solutions graphiques du TD n°1 . . . 13

3 Conclusion 14

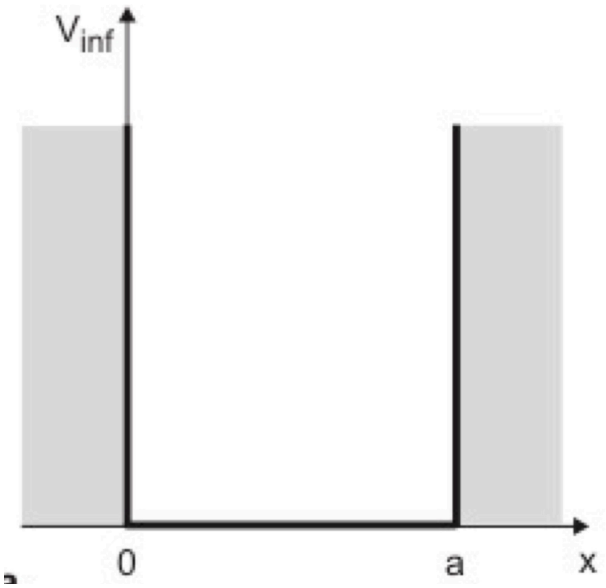
4 Annexes 14
4.1 Scripts Partie I . . . 14
4.2 Scripts Partie II . . . 16

1 L'oscillateur harmonique dans tous ses états (quantiques)

1.1 Étude préliminaire

1.1.1 Puits de potentiel infini

Considérons une particule de masse m confinée dans le puits de potentiel infini ci-dessous :



L'hamiltonien du système en représentation |x> est rappelé dans le sujet :

$$H_0 = \frac{-\hbar^2}{2.m}\frac{d^2}{dx^2} + V_{inf}(x)$$

Avec $V_{inf}(x) = 0$ si $0 < x < a$ et $V_{inf}(x) = +\infty$ sinon
On cherche donc à résoudre l'équation aux valeurs propres suivante :

$$H_0\varphi = E\varphi$$

Condition aux limites : la fonction d'onde doit être continue donc $\varphi(x = 0) = 0$ et $\varphi(x = a) = 0$
On se place dans le cas où $E > 0$ car si $E \leq 0$ il n'y a pas de quanton possible. Dans le puits en représentation |x>, on a :

$$\frac{d^2\varphi}{dx^2} + \frac{2.m.E}{\hbar^2}\varphi = 0$$

Posons $k = \frac{\sqrt{2.m.E}}{\hbar}$, on a alors :

$$\varphi(x) = A.\cos(k.x) + B.\sin(k.x)$$

D'après les conditions aux limites :

$$\varphi(0) = A = 0$$

$$\varphi(a) = B.sin(k.a) = 0 \Rightarrow k_n.a = n.\pi \Rightarrow k_n = \frac{n.\pi}{a}$$

Ainsi :

$$E_n = \frac{n^2.\pi^2.\hbar^2}{2.m.a^2}$$

Déterminons B :

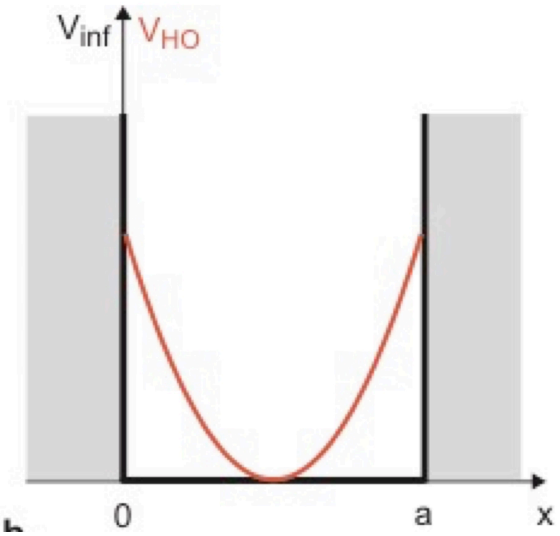
$$\int_0^a |\varphi_n(x)|^2.dx = 1 \Rightarrow B^2.\int_0^a \sin(k.x)^2.dx = 1 \Rightarrow B.\sqrt{\frac{a}{2}} = 1 \Rightarrow B = \sqrt{\frac{2}{a}}$$

Finalement :

$$\varphi_n(x) = \sqrt{\frac{2}{a}}.\sin(\frac{n.\pi.x}{a})$$

1.1.2 Potentiel harmonique dans un puits infini

Considérons maintenant le cas d'un potentiel harmonique localisé à l'intérieur du puits infini :



L'hamiltonien du système en représentation |x> est rappelé dans le sujet :

$$H = H_0 + V_{OH}$$

où $V_{OH} = \frac{1}{2}.m.\omega^2(x - \frac{a}{2})^2$

Posons $|\Psi >= \sum_{m=1}^{+\infty} c_m.|\varphi_m >$, en réalisant le produit scalaire entre l'équation de Schrödinger $H|\Psi >= E|\Psi >$ avec un bra $< \varphi_n|$, on a :

$$< \varphi_n|H|\Psi >= E < \varphi_n|\Psi >= E.c_n$$

Or en exploitant la représentation matricielle de l'hamiltonien, on a :

$$H|\Psi >= \sum_{k=1}^{+\infty}\sum_{m=1}^{+\infty} H_{k,m}c_m|\varphi_k >$$

$$\Rightarrow < \varphi_n|H|\Psi >= \sum_{k=1}^{+\infty}\sum_{m=1}^{+\infty} H_{k,m}c_m < \varphi_n|\varphi_k >= \sum_{m=1}^{+\infty} H_{n,m}c_m$$

D'où l'équation suivante valable pour tout entier naturel n non nul :

$$\sum_{m=1}^{+\infty} H_{n,m}c_m = E.c_n$$

1.2 Potentiel normalisé et solutions

a. Exprimons le potentiel harmonique sous forme normalisée :

$$\nu_{OH} = \frac{V_{OH}}{E_1^{(0)}} = \frac{1}{(E_1^{(0)})^2}\frac{1}{2}.m.\omega^2(x - \frac{a}{2})^2.\frac{\pi^2.\hbar^2}{2.m.a^2} = \frac{\pi^2}{4}.(\frac{\hbar.\omega}{E_1^{(0)}})^2.(\frac{x}{a} - \frac{1}{2})^2$$

b. Exprimons les coefficients de **H** normalisés :

$$h_{n,m} = \frac{H_{n,m}}{E_1^{(0)}} = n^2.\delta_{n,m} + \frac{2}{a}.\int_0^1 \sin(\frac{n.\pi.x}{a}).\frac{V_{OH}}{E_1^{(0)}}.\sin(\frac{m.\pi.x}{a}).dx$$

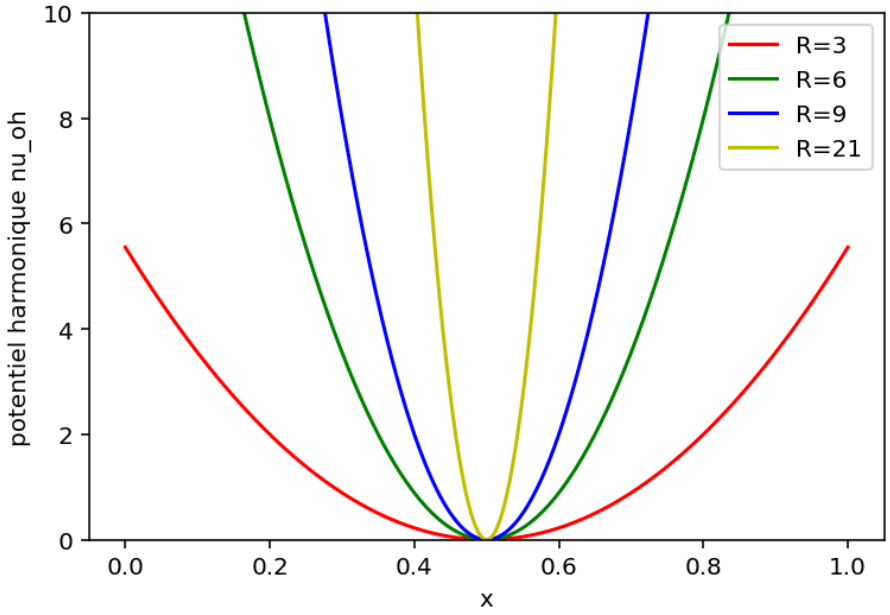
Soit en posant $x' = \frac{x}{a}$ avec $dx' = \frac{dx}{a}$:

$$h_{n,m} = n^2.\delta_{n,m} + 2.\int_0^1 \sin(n.\pi.x').\nu_{OH}.\sin(m.\pi.x').dx'$$

c. Exprimons ν_{OH} en fonction de R :

$$\nu_{OH} = \frac{\pi^2}{4}.R^2.(\frac{x}{a} - \frac{1}{2})^2$$

On peut alors tracer le potentiel harmonique pour différentes valeurs de R :



1.3 Résolution numérique

1.3.1 Symétries de la matrice H

J'ai arbitrairement choisi d'afficher **H** avec des coefficients précis à 10^{-3} pour simplifier la lecture :

$$\mathbf{H} = \begin{bmatrix} 73.555 & 0.000 & 84.375 & 0.000 & 15.625 & 0.000 \\ 0.000 & 160.930 & 0.000 & 100.000 & 0.000 & 21.094 \\ 84.375 & 0.000 & 181.555 & 0.000 & 105.469 & 0.000 \\ 0.000 & 100.000 & 0.000 & 194.024 & 0.000 & 108.000 \\ 15.625 & 0.000 & 105.469 & 0.000 & 205.555 & 0.000 \\ 0.000 & 21.094 & 0.000 & 108.000 & 0.000 & 217.930 \end{bmatrix}$$

On constate que **H** est symétrique et que si n et m ne sont pas de même parité $H_{n,m} = 0$, montrons cela :

$$h_{n,m} = n^2.\delta_{n,m} + 2.\int_0^1 \sin(n.\pi.x).\nu_{OH}(x).\sin(m.\pi.x).dx = n^2.\delta_{n,m} + \int_0^1 \nu_{OH}(x).(\cos((n-m).\pi.x) - \cos((n+m).\pi.x)).dx$$

Si $n \neq m$, on a :

$$h_{n,m} = \int_0^1 \nu_{OH}(x).(\cos((n-m).\pi.x) - \cos((n+m).\pi.x)).dx = \int_0^1 \nu_{OH}(x).(\cos((m-n).\pi.x) - \cos((n+m).\pi.x)).dx = h_{m,n}$$

Grâce à la parité de la fonction cosinus on a bien **H** symétrique.

$$h_{n,m} = \frac{\pi^2.R^2}{4}\int_0^1 (x - \frac{1}{2})^2.(\cos((n-m).\pi.x) - \cos((n+m).\pi.x)).dx$$

Posons maintenant (pour $k \neq 0$) :

$$I_k = \frac{\pi^2.R^2}{4} \int_0^1 (x - \frac{1}{2})^2 . \cos(k.\pi.x).dx$$

En réalisant deux intégrations par parties successives on a :

$$\begin{aligned}
I_k &= \frac{\pi^2 R^2}{4} \int_0^1 \left(x - \frac{1}{2}\right)^2 \cos(k\pi x)\, dx \\
&= \frac{\pi^2 R^2}{4} \left[\left[\left(x - \frac{1}{2}\right)^2 \frac{\sin(k\pi x)}{k\pi} \right]_0^1 - \int_0^1 2 \left(x - \frac{1}{2}\right) \frac{\sin(k\pi x)}{k\pi}\, dx \right] \\
&= \frac{\pi^2 R^2}{4} \left(-2 \int_0^1 \left(x - \frac{1}{2}\right) \frac{\sin(k\pi x)}{k\pi}\, dx \right) \\
&= \frac{\pi^2 R^2}{4} \left(2 \left[\left(x - \frac{1}{2}\right) \frac{\cos(k\pi x)}{(k\pi)^2} \right]_0^1 - \int_0^1 \frac{\cos(k\pi x)}{(k\pi)^2}\, dx \right) \\
&= \frac{\pi^2 R^2}{2} \left[\frac{1}{2} \frac{1}{(k\pi)^2} (-1)^k + \frac{1}{2} \frac{1}{(k\pi)^2} \right] \\
&= \frac{R^2}{4.k^2} \left(1 + (-1)^k\right).
\end{aligned}$$

On en déduit que $\forall n, m \ / \ n \neq m$:

$$h_{n,m} = I_{n-m} - I_{n+m} = \begin{cases} 0 & \text{si } n \text{ et } m \text{ ne sont pas de même parité,} \\ \dfrac{R^2}{2} \dfrac{4nm}{(n^2 - m^2)^2} & \text{sinon.} \end{cases}$$

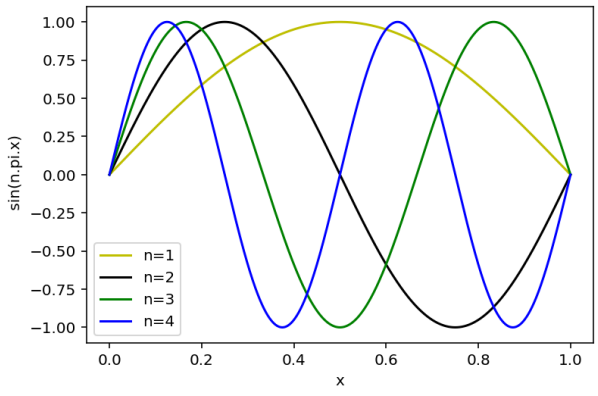
et si $n = m$:

$$h_{n,n} = n^2 + \frac{\pi^2.R^2}{4} \int_0^1 (x - \frac{1}{2})^2 .dx - I_{2n} = n^2 + \frac{\pi^2.R^2}{4} \frac{1}{12} - \frac{R^2}{8.n^2}$$

Soit

$$h_{n,n} = n^2 + \frac{R^2}{8} (\frac{\pi^2}{6} - \frac{1}{n^2})$$

Interprétation physique : $h_{n,m}$ représente le couplage énergétique entre le vecteur $|\varphi_n\rangle$ et le vecteur $|\varphi_m\rangle$. Lorsque ce terme est nul on a une énergie d'interaction nulle entre ces deux états. Or $\varphi_n(x)$ est proportionnelle à $\sin(n\pi x)$ et cette fonction est paire (respectivement impaire) par rapport à l'axe verticale passant par x=0.5, si n est impaire (respectivement paire). On le constate sur le graphique suivant :

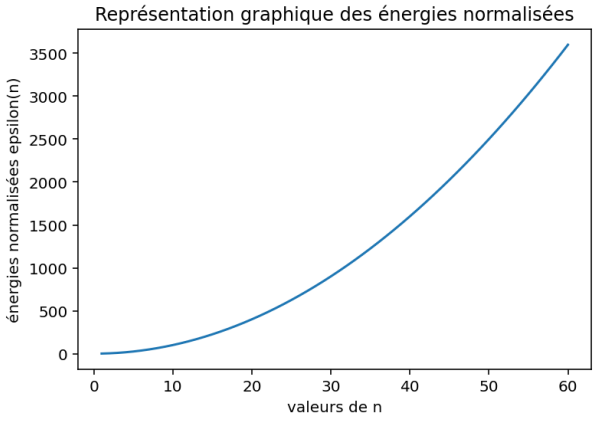


Ainsi les fonctions paires et impaires par rapport à l'axe verticale passant par x=0.5 sont découplées, on peut donc distinguer deux classes de vecteurs.

1.3.2 Energies propres

N est fixé à 60 et nous allons déterminer numériquement les énergies propres normalisées associées au système : $\epsilon(n) = E(n)/E_1^0$. Comme il est rappelé dans l'énoncé, l'objectif de cette partie est de déterminer les énergies propres d'une particule soumise à un potentiel harmonique tronqué par un puits infini. On comprend donc le sens de la comparaison des $\epsilon(n)$ avec les énergies normalisées théoriques d'un potentiel harmonique.

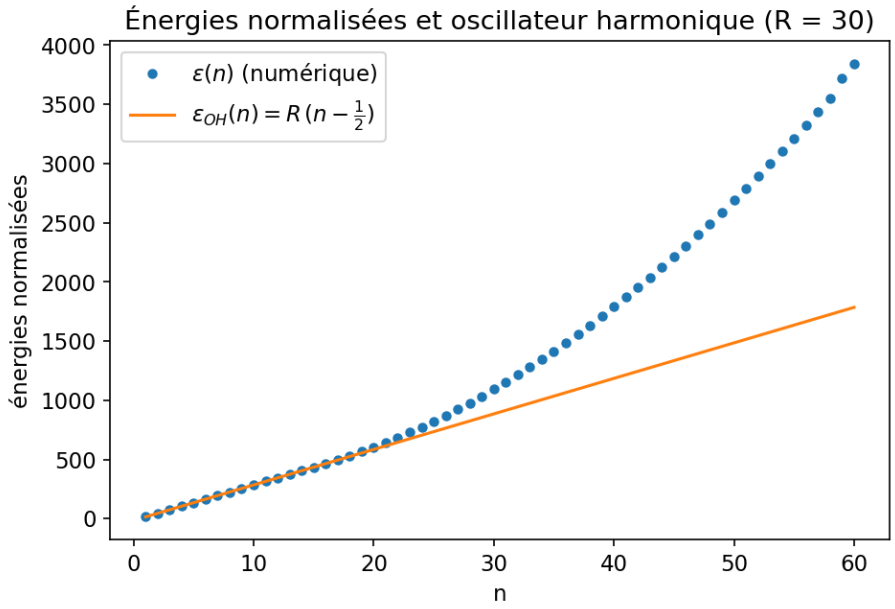
a. En calculant numériquement les valeurs de $\epsilon(n)$ et en les traçant en fonction de n, on obtient :



b. Les énergies propres de l'oscillateur harmonique non tronqué s'écrivent $E_{OH}(k) = \hbar\omega(k + \frac{1}{2})$ avec $k = 0, 1, 2, \ldots$ On les normalise par $E_1^{(0)}$ pour comparer des choses comparables. Comme $R = \frac{\hbar\omega}{E_1^{(0)}}$ et que l'indice n commence à 1 (donc $k = n - 1$), on obtient :

$$\epsilon_{OH}(n) = \frac{E_{OH}(n-1)}{E_1^{(0)}} = R\left(n - \frac{1}{2}\right)$$

En traçant $\epsilon(n)$ et $\epsilon_{OH}(n)$ sur la même figure, on obtient :



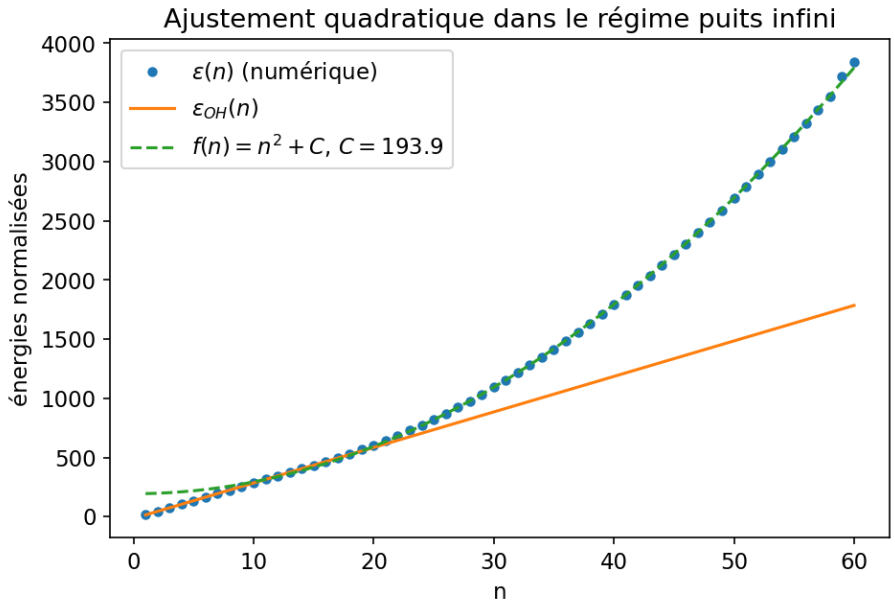
Pour les premiers niveaux, l'accord est excellent : on trouve numériquement $\epsilon(1) = 15$, $\epsilon(2) = 45$, $\epsilon(3) = 75$, $\epsilon(4) = 105$, $\epsilon(5) = 135$, soit exactement $R(n - \frac{1}{2})$. L'écart relatif reste inférieur à $0,01\,\%$ jusqu'à $n = 12$ et à

1 % jusqu'à $n = 17$, puis les deux courbes se séparent.

c. On constate également que pour n grand $\epsilon(n)$ change d'allure et devient quadratique et on souhaite montrer que les énergies propres sont alors bien ajustées par une courbe de la forme $f(n) = n^2 + C$. On détermine C en minimisant l'erreur quadratique $\sum_n \left(\epsilon(n) - n^2 - C\right)^2$ sur les niveaux du régime « puits infini », c'est-à-dire pour $n > n_0$ (voir question d.) :

$$C = \frac{1}{N - n_0} \sum_{n=n_0+1}^{N} \left(\epsilon(n) - n^2\right)$$

Voir le calcul de C dans le code Python en annexe.
On obtient alors la représentation graphique suivante :



La valeur de C déterminée est : $C = 193,9$. L'écart relatif entre $\epsilon(n)$ et $f(n)$ reste inférieur à 2 % sur tout le régime $n > n_0$.

Mais d'où provient C ?
Rappelons que l'équation (8) nous a permis d'exprimer littéralement le coefficient $h_{n,n}$:

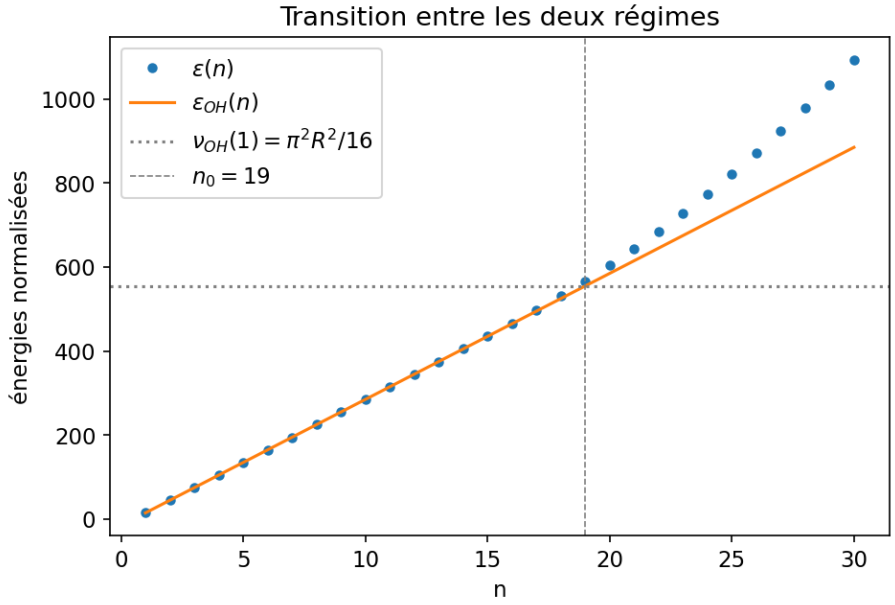
$$h_{n,n} = n^2 + \frac{R^2}{8}\left(\frac{\pi^2}{6} - \frac{1}{n^2}\right)$$

Or pour n très grand le terme en $\frac{1}{n^2}$ est négligeable et on a une expression des coefficients diagonaux :

$$h_{n,n} \underset{n \to +\infty}{\approx} n^2 + \frac{\pi^2.R^2}{48}$$

De plus les coefficients diagonaux sont grands devant les coefficients non diagonaux, on peut donc considérer que les valeurs propres ont le même comportement que les coefficients diagonaux. On a donc trouvé l'origine de C : $C(R) \approx \frac{\pi^2.R^2}{48} \approx 185,1$ pour $R = 30$. L'écart d'environ 5 % avec la valeur ajustée provient des termes non diagonaux, négligés dans cette estimation.

d. On peut visualiser la transition entre les deux régimes à l'aide de la figure ci-dessous :



Lorsque l'énergie normalisée de la particule dépasse la valeur du potentiel normalisé de l'oscillateur harmonique au bord du puits, celle-ci est contrainte par les parois du puits infini. Le potentiel harmonique est alors négligeable devant le potentiel du puits infini et on observe la transition de régime.
Notons que $\nu_{OH}(1) = \frac{\pi^2.R^2}{16} = 555,165$ qui correspond donc à l'énergie limite de la particule à partir de laquelle on observe la transition de régime. En comparant cette valeur aux valeurs $\epsilon(n)$, on trouve $n_0 = 19$ avec $\epsilon(n_0) = 566,873$, premier niveau d'énergie supérieure à $\nu_{OH}(1)$.

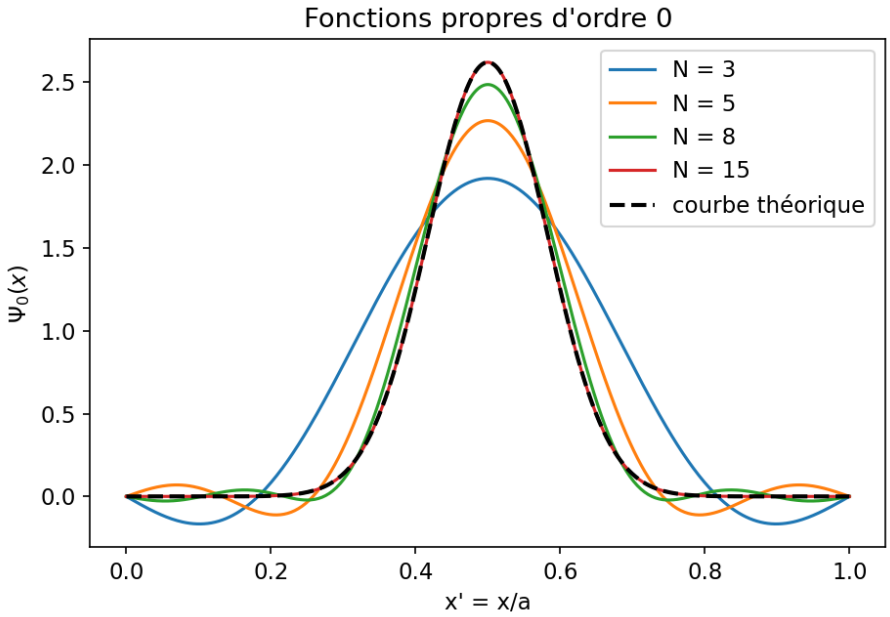
1.3.3 Fonctions propres

On prend maintenant a=1 et on va étudier les fonctions propres de la particule.

a. Notons qu'on peut exprimer les fonctions propres dans la base constituée par les fonctions φ_n définies dans le préliminaire :

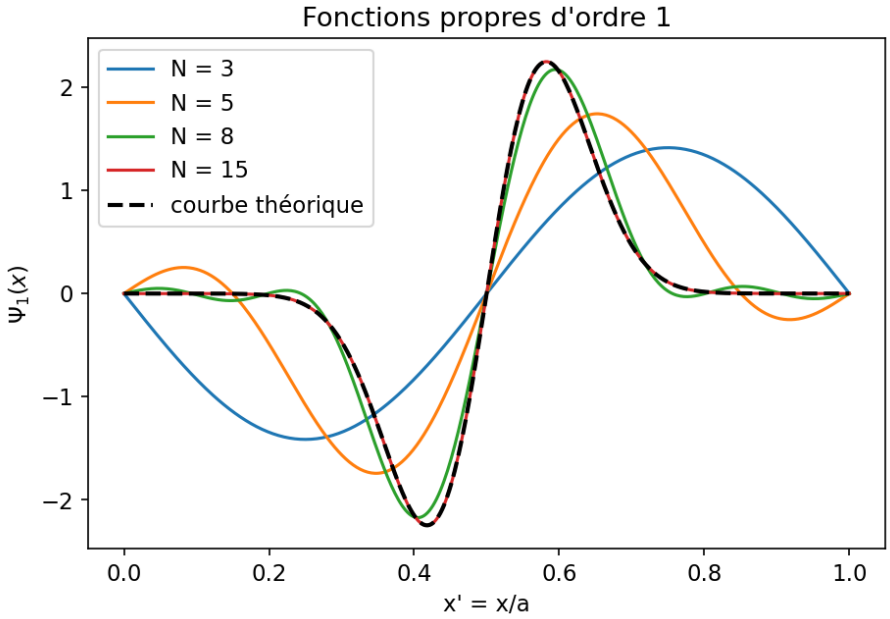
$$\Psi_n(x) = \sum_{i=1}^{N} c_i^n.\varphi_i(x)$$

Où les coefficients c_i^n correspondent à la i-ème coordonnée de Ψ_n dans la base φ_i, c'est-à-dire à la i-ème composante du vecteur propre de $\mathbf{H}$ associé à la n-ième plus petite valeur propre.
On peut donc déterminer numériquement la fonction d'onde de l'état fondamental $\Psi_0(x)$ pour différentes valeurs de N et lorsqu'on les trace avec la fonction théorique de l'énoncé on obtient :



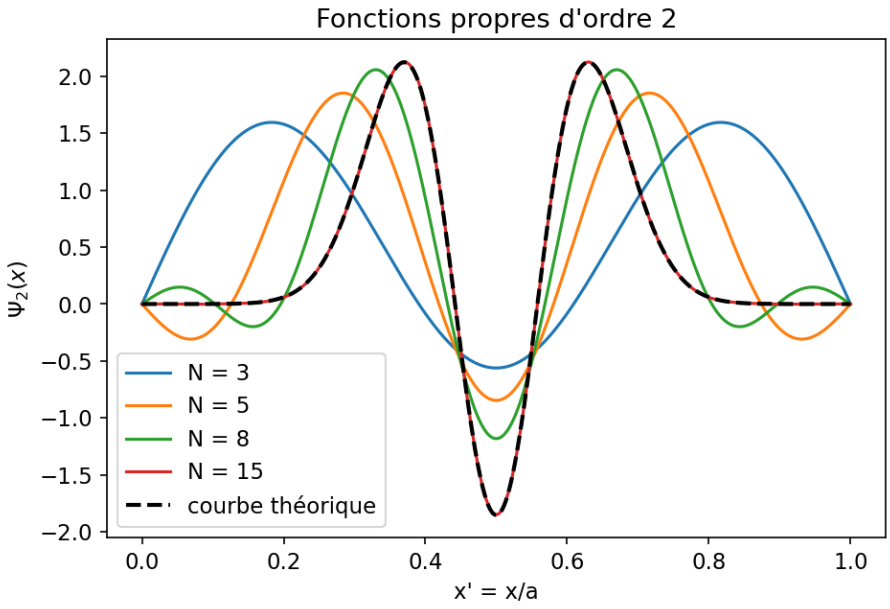
On remarque que plus N est grand plus la correspondance avec la fonction théorique est bonne : pour $N = 15$, la courbe numérique est confondue avec la courbe théorique.

b. En faisant la même chose avec l'état $\Psi_1(x)$, on obtient :



À nouveau on remarque que plus N est grand plus le modèle théorique est pertinent.

c. Pour l'état $\Psi_2(x)$, on obtient :

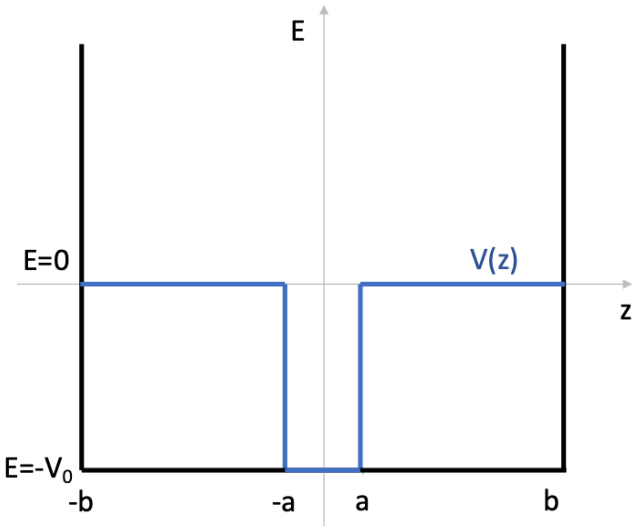


Et on fait à nouveau le constat que plus N est grand plus nos courbes s'approchent de la représentation théorique. Les états excités, plus étalés et plus oscillants, nécessitent davantage de fonctions de base que l'état fondamental pour être décrits correctement.

2 Puits de potentiel fini dans les hétérostructures de semi-conducteurs

2.1 Modèle du puits de potentiel

On considère la situation présentée sur la figure ci-dessous où le puits de potentiel fini qui décrit la structure $Al_xGa_{1-x}As/GaAs/Al_xGa_{1-x}As$ a une largeur $2a$ et une profondeur $-V_0$. Ce puits de potentiel est tronqué par un puits infini de largeur $2b$ et de profondeur $-V_0$.



L'hamiltonien du système est :

$$H = H_0 + V(z)$$

où $H_0 = \frac{p^2}{2m} - V_0$ correspond à l'hamiltonien du puits infini. On notera la présence du potentiel $-V_0$ qui implique que les énergies des modes propres sont décalées de $-V_0$ par rapport aux solutions du puits infini

vu en cours et où $V(z) = 0$ si $|z| < a$ et $V(z) = V_0$ si $b > |z| > a$. La masse m est la masse d'un électron et on prendra les paramètres suivants : a=1nm et $V_0 = 0.5$eV. Le paramètre b sera choisi suffisamment grand pour assurer la décroissance des fonctions d'onde en dehors du puits fini, par exemple b=5a.

2.2 Résolution numérique

2.2.1 Base de fonctions propres

On obtient les fonctions propres du cas présentés ci-dessus en reprenant les fonctions trouvées au I.A) et en posant $a = 2b$ et $x = z + b$ de plus les énergies propres sont simplement abaissées de $-V_0$ d'où :

$$\varphi_n(z) = \sqrt{\frac{1}{b}} \sin(\frac{n.\pi}{2b}(z+b))$$

$$E_n = \frac{n^2.\hbar^2.\pi^2}{8.m.b^2} - V_0$$

2.2.2 Solutions numériques

Pour déterminer les états confinés du puits de potentiel fini, nous allons appliquer la même méthode numérique que dans la partie I, en développant la fonction d'onde sur la base des états propres du puits infini tronqué de largeur $2b$. Les fonctions propres utilisées sont les φ_n déterminées ci-dessus tout comme les énergies correspondantes

La matrice de l'Hamiltonien dans cette base s'écrit alors :

$$H_{nm} = E_n^{(0)}\,\delta_{nm} + \int_{-b}^{b} \varphi_n(z)\,V(z)\,\varphi_m(z)\,dz,$$

où

$$V(z) = \begin{cases} 0 & \text{si } |z| < a, \\ V_0 & \text{si } a < |z| < b. \end{cases}$$

La matrice H a été construite numériquement avec une base de taille $N = 150$. La diagonalisation de H fournit l'ensemble des valeurs propres.
Après avoir trié les valeurs propres de H, j'ai affiché les 5 plus basses énergies :

$$E_0 = -0,4428 \text{ eV}, \qquad E_1 = -0,2782 \text{ eV}, \qquad E_2 = -0,0463 \text{ eV}, \qquad E_3 = 0,0216 \text{ eV}, \qquad E_4 = 0,0289 \text{ eV}$$

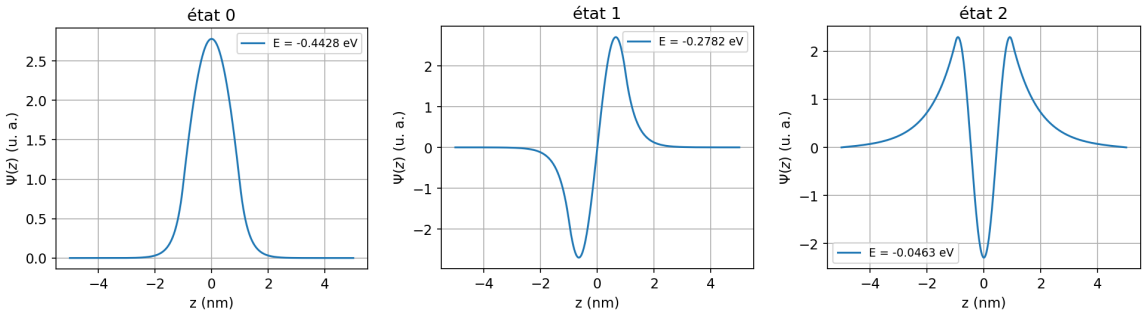
Les énergies négatives correspondent aux états confinés dans le puits fini, et nous trouvons trois états confinés, en accord avec le résultat attendu :

$$E_0 = -0,4428 \text{ eV}, \qquad E_1 = -0,2782 \text{ eV}, \qquad E_2 = -0,0463 \text{ eV}$$

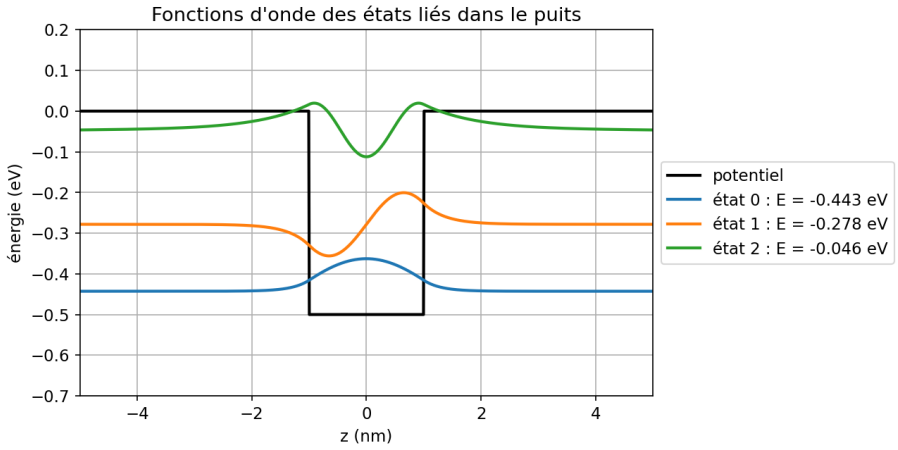
Les fonctions d'onde associées sont reconstruites via :

$$\Psi_n(z) = \sum_{k=1}^{N} c_k^{(n)}\,\varphi_k(z),$$

Après les avoir normalisées, on peut les tracer :



Leur tracé superposé au potentiel $V(z)$ donne :



On retrouve la structure donnée pour un puits fini dans le sujet.

2.2.3 Lien avec les solutions graphiques du TD n°1

Dans le TD n°1, nous avions montré que les niveaux d'énergie d'un puits fini sont obtenus à partir des équations suivantes :

$$|\cos(q_n a)| = \frac{q_n}{q} \qquad \text{(états pairs)},$$

$$|\sin(q_n a)| = \frac{q_n}{q} \qquad \text{(états impairs)},$$

où

$$V_0 = \frac{\hbar^2 q^2}{2m}, \qquad E_n = -\frac{\hbar^2 k_n^2}{2m}.$$

Avec $q^2 = q_n^2 + k_n^2$
On peut alors exprimer k_n, et q (avec $m = 9,11.10^{-31}$ kg) :

$$k_n = \sqrt{\frac{-2mE_n}{\hbar^2}}, \qquad q = \sqrt{\frac{2mV_0}{\hbar^2}} = 3,62.10^9 \text{m}^{-1}$$

La représentation des fonctions d'onde des états liés dans le puits montre que : l'état 0 est paire, l'état 1 est impaire et l'état 2 est paire.
Calculons maintenant les différentes valeurs de k_n et q_n :

Pour l'état 0 :

$$k_0 = 3,41.10^9 \text{m}^{-1}, \qquad q_0 = 1,23.10^9 \text{m}^{-1}$$

D'où : $|\cos q_0 a| = 0,338$ et $\frac{q_0}{q} = 0,338$

Pour l'état 1 :

$$k_1 = 2,70.10^9 \text{m}^{-1}, \qquad q_1 = 2,41.10^9 \text{m}^{-1}$$

D'où : $|\sin q_1 a| = 0,666$ et $\frac{q_1}{q} = 0,666$

Pour l'état 2 :

$$k_2 = 1,10.10^9 \text{m}^{-1}, \qquad q_2 = 3,45.10^9 \text{m}^{-1}$$

D'où : $|\cos q_2 a| = 0,953$ et $\frac{q_2}{q} = 0,953$

Cette excellente concordance démontre que :

1. la méthode numérique matricielle reproduit fidèlement les conditions de quantification du puits de potentiel fini,
2. les trois niveaux négatifs trouvés correspondent bien aux seules solutions physiquement confinées dans le puits GaAs.

Ainsi, les solutions numériques et les solutions graphiques du TD sont parfaitement cohérentes (au millième près).

3 Conclusion

Ce travail nous a donc permis d'obtenir des solutions numériques approchées pour l'oscillateur harmonique quantique et de les comparer aux solutions analytiques établies en cours, validant ainsi de manière rigoureuse nos résultats. Nous avons mis en évidence la signification des états confinés et leur représentation, notions qui restent difficiles à appréhender sans visualisation directe.

4 Annexes

4.1 Scripts Partie I

```
#!/usr/bin/env python3
"""
Oscillateur harmonique quantique tronqué par un puits infini (méthode de Marsiglio).

On développe la fonction d'onde sur la base des états propres du puits infini
phi_n(x) = sqrt(2) sin(n pi x), 0 < x < 1, et on diagonalise la matrice de
l'hamiltonien normalisé  h_nm = n^2 delta_nm + 2 int sin(n pi x) nu_OH(x) sin(m pi x) dx
avec nu_OH(x) = (pi^2 / 4) R^2 (x - 1/2)^2  et  R = hbar omega / E_1.

Référence : F. Marsiglio, "The harmonic oscillator in quantum mechanics: A third way",
Am. J. Phys. 77, 253 (2009).

Auteur : Maxime Rimbaud - ESPCI Paris - PSL, TP numérique de physique quantique (2025).
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import quad

FIG = Path(__file__).parent / "figures"
FIG.mkdir(exist_ok=True)
plt.rcParams.update({"font.size": 11, "figure.dpi": 120})

R = 30          # rapport hbar*omega / E_1 imposé par l'énoncé
N = 60          # taille de la base pour l'étude des énergies


def nu_oh(x, R):
    """Potentiel harmonique normalisé par E_1, centré en x = 1/2."""
    return (np.pi**2 / 4) * R**2 * (x - 0.5) ** 2


def hamiltonien(R, N):
    """Matrice h (N x N) calculée par intégration numérique."""
    h = np.zeros((N, N))
    for n in range(1, N + 1):
        for m in range(n, N + 1):
```

```
            integrale = quad(lambda x: np.sin(n * np.pi * x) * np.sin(m * np.pi * x) * nu_oh(x,
                R), 0, 1)[0]
            h[n - 1, m - 1] = h[m - 1, n - 1] = (n**2 if n == m else 0) + 2 * integrale
    return h


def hamiltonien_analytique(R, N):
    """Même matrice à partir des expressions analytiques démontrées dans le rapport."""
    n = np.arange(1, N + 1)
    nn, mm = np.meshgrid(n, n, indexing="ij")
    meme_parite = (nn - mm) % 2 == 0
    with np.errstate(divide="ignore", invalid="ignore"):
        h = np.where(meme_parite, 2 * R**2 * nn * mm / (nn**2 - mm**2) ** 2, 0.0)
    np.fill_diagonal(h, n**2 + R**2 / 8 * (np.pi**2 / 6 - 1 / n**2))
    return h


def etats_propres(R, N):
    """Énergies propres triées et vecteurs propres associés (en colonnes)."""
    energies, vecteurs = np.linalg.eigh(hamiltonien(R, N))
    return energies, vecteurs


def fonction_onde(coefficients, x):
    """Psi(x) = sum_j c_j sqrt(2) sin(j pi x)."""
    j = np.arange(1, len(coefficients) + 1)
    return np.sqrt(2) * np.einsum("ij,j->i", np.sin(np.outer(x, j) * np.pi), coefficients)


# --- Solutions analytiques de l'oscillateur harmonique (a = 1) -------------------------
def psi0_th(x, R):
    return (np.pi * R / 2) ** 0.25 * np.exp(-np.pi**2 * R * (x - 0.5) ** 2 / 4)


def psi1_th(x, R):
    return (np.pi**5 * R**3 / 2) ** 0.25 * (x - 0.5) * np.exp(-np.pi**2 * R * (x - 0.5) ** 2 / 4)


def psi2_th(x, R):
    return (np.pi * R / 8) ** 0.25 * (np.pi**2 * R * (x - 0.5) ** 2 - 1) * np.exp(-np.pi**2 * R *
        (x - 0.5) ** 2 / 4)


if __name__ == "__main__":
    # 1) Potentiel normalisé pour plusieurs valeurs de R
    x = np.linspace(0, 1, 1000)
    plt.figure()
    for r in (3, 6, 9, 21):
        plt.plot(x, nu_oh(x, r), label=f"R = {r}")
    plt.ylim(0, 10)
    plt.xlabel("x / a")
    plt.ylabel(r"$\nu_{OH}(x)$")
    plt.title("Potentiel harmonique normalisé dans le puits infini")
    plt.legend()
    plt.savefig(FIG / "potentiel_harmonique.png", bbox_inches="tight")
    plt.close()

    # 2) Matrice H pour N = 6 et vérification par les formules analytiques
    h6 = hamiltonien(R, 6)
    print("Matrice h normalisée (N = 6, R = 30) :")
    print(np.array2string(h6, precision=3, suppress_small=True))
    ecart = np.abs(hamiltonien(R, N) - hamiltonien_analytique(R, N)).max()
    print(f"Écart max. intégration numérique / formules analytiques (N = {N}) : {ecart:.1e}")

    # 3) Énergies propres normalisées et comparaison avec l'oscillateur harmonique
    eps, _ = etats_propres(R, N)
    n = np.arange(1, N + 1)
    eps_oh = R * (n - 1 + 0.5)                      # E_OH(k) / E_1 = R (k + 1/2), k = n - 1
```

```
    n0 = int(np.argmax(eps > nu_oh(1.0, R))) + 1    # premier niveau au-dessus du bord du puits
    regime_puits = n > n0
    C = np.mean(eps[regime_puits] - n[regime_puits] ** 2)   # moindres carrés pour f(n) = n^2 + C

    print(f"\n5 premières énergies normalisées : {np.round(eps[:5], 4)}")
    print(f"Oscillateur harmonique R(k+1/2)  : {eps_oh[:5]}")
    print(f"Transition de régime : nu_OH(1) = {nu_oh(1.0, R):.1f}, n0 = {n0}, eps(n0) = {eps[n0 - 1]:.1f}")
    print(f"Ajustement f(n) = n^2 + C sur n > n0 : C = {C:.1f}  (estimation diagonale pi^2 R^2 / 48 = {np.pi**2 * R**2 / 48:.1f})")

    plt.figure()
    plt.plot(n, eps, "o", ms=4, label=r"$\varepsilon(n)$ numérique")
    plt.plot(n, eps_oh, "-", label=r"oscillateur harmonique $R(k+\frac{1}{2})$")
    plt.plot(n, n**2 + C, "--", label=rf"$n^2 + C$, $C = {C:.0f}$")
    plt.axhline(nu_oh(1.0, R), color="grey", ls=":", label=r"bord du puits $\nu_{OH}(1)$")
    plt.axvline(n0, color="grey", ls=":")
    plt.xlabel("n")
    plt.ylabel(r"$\varepsilon(n) = E_n / E_1$")
    plt.title(f"Énergies propres normalisées (R = {R}, N = {N})")
    plt.legend()
    plt.savefig(FIG / "energies_propres.png", bbox_inches="tight")
    plt.close()

    # 4) Convergence des fonctions propres avec la taille de la base
    x = np.linspace(0, 1, 400)
    theoriques = (psi0_th, psi1_th, psi2_th)
    fig, axes = plt.subplots(1, 3, figsize=(14, 4), sharey=True)
    for etat, (ax, psi_th) in enumerate(zip(axes, theoriques)):
        reference = psi_th(x, R)
        for taille in (3, 5, 8, 15):
            _, vecteurs = etats_propres(R, taille)
            psi = fonction_onde(vecteurs[:, etat], x)
            psi *= np.sign(psi @ reference)          # phase globale arbitraire
            ax.plot(x, psi, label=f"N = {taille}")
        ax.plot(x, reference, "k--", lw=2, label="théorie")
        ax.set_title(rf"$\Psi_{etat}(x)$")
        ax.set_xlabel("x / a")
    axes[0].legend()
    fig.suptitle(f"Convergence des fonctions propres avec la taille de la base (R = {R})")
    fig.savefig(FIG / "fonctions_propres.png", bbox_inches="tight")
    plt.close(fig)

    print(f"\nFigures enregistrées dans {FIG}")
```

4.2 Scripts Partie II

```
#!/usr/bin/env python3
"""
États liés d'un puits quantique fini (hétérostructure AlGaAs / GaAs / AlGaAs).

Puits de largeur 2a et de profondeur V0, tronqué par un puits infini de largeur 2b.
On diagonalise l'hamiltonien dans la base des états propres du puits infini
phi_n(z) = sin(n pi (z + b) / 2b) / sqrt(b), puis on vérifie les conditions de
quantification |cos(q a)| = q / q0 (états pairs) et |sin(q a)| = q / q0 (états impairs).

Auteur : Maxime Rimbaud - ESPCI Paris - PSL, TP numérique de physique quantique (2025).
"""

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from scipy.constants import electron_volt as eV, hbar, m_e
from scipy.integrate import quad
```

```python

FIG = Path(__file__).parent / "figures"
FIG.mkdir(exist_ok=True)
plt.rcParams.update({"font.size": 11, "figure.dpi": 120})

a = 1e-9        # demi-largeur du puits fini (m)
b = 5 * a       # demi-largeur du puits infini (m)
V0 = 0.5        # profondeur du puits (eV)
N = 150         # taille de la base


def phi(n, z):
    return np.sin(n * np.pi * (z + b) / (2 * b)) / np.sqrt(b)


def energie_puits_infini(n):
    """Énergie du n-ième état du puits infini de largeur 2b, décalée de -V0 (eV)."""
    return (n * np.pi * hbar) ** 2 / (8 * m_e * b**2) / eV - V0


def hamiltonien(N):
    """H_nm = E_n delta_nm + int phi_n V phi_m, avec V = +V0 pour a < |z| < b (barrières)."""
    H = np.diag([energie_puits_infini(n) for n in range(1, N + 1)])
    for n in range(1, N + 1):
        for m in range(n, N + 1):
            if (n + m) % 2:                    # couplage nul entre états de parités différentes
                continue
            barrieres = sum(quad(lambda z: phi(n, z) * phi(m, z), zmin, zmax, limit=200)[0]
                            for zmin, zmax in ((-b, -a), (a, b)))
            H[n - 1, m - 1] += V0 * barrieres
            if m != n:
                H[m - 1, n - 1] = H[n - 1, m - 1]
    return H


if __name__ == "__main__":
    energies, vecteurs = np.linalg.eigh(hamiltonien(N))
    lies = np.where(energies < 0)[0]
    print(f"5 énergies propres les plus basses (eV) : {np.round(energies[:5], 4)}")
    print(f"Nombre d'états liés (E < 0) : {len(lies)}")

    z = np.linspace(-b, b, 1000)
    fonctions = []
    for k in lies:
        psi = sum(vecteurs[n - 1, k] * phi(n, z) for n in range(1, N + 1))
        psi /= np.sqrt(np.trapezoid(psi**2, z))
        fonctions.append(psi)

    # Vérification des conditions de quantification du puits fini (TD n°1)
    q0 = np.sqrt(2 * m_e * V0 * eV) / hbar
    print(f"\nq0 = {q0:.3e} m^-1")
    for k, E in zip(lies, energies[lies]):
        q = np.sqrt(2 * m_e * (E + V0) * eV) / hbar      # vecteur d'onde dans le puits
        pair = k % 2 == 0
        membre_gauche = abs(np.cos(q * a)) if pair else abs(np.sin(q * a))
        print(f"État {k} ({'pair' if pair else 'impair'}) : E = {E:+.4f} eV, "
              f"|{'cos' if pair else 'sin'}(qa)| = {membre_gauche:.3f}, q/q0 = {q / q0:.3f}")

    # Figure : potentiel, niveaux et fonctions d'onde (amplitude arbitraire)
    potentiel = np.where(np.abs(z) < a, -V0, 0.0)
    plt.figure(figsize=(7, 5))
    plt.plot(z * 1e9, potentiel, "k", lw=2, label="V(z)")
    echelle = 0.08 / max(np.abs(f).max() for f in fonctions)
    for k, psi in zip(lies, fonctions):
        plt.axhline(energies[k], color="grey", ls=":", lw=0.8)
        plt.plot(z * 1e9, energies[k] + echelle * psi, lw=2, label=f"état {k} : E =
            {energies[k]:.3f} eV")
    plt.xlim(-b * 1e9, b * 1e9)
```

```
    plt.ylim(-0.7, 0.2)
    plt.xlabel("z (nm)")
    plt.ylabel("Énergie (eV)")
    plt.title("États liés du puits fini GaAs (a = 1 nm, V0 = 0,5 eV)")
    plt.legend(loc="lower right", fontsize=9)
    plt.savefig(FIG / "puits_fini_etats_lies.png", bbox_inches="tight")
    plt.close()
    print(f"\nFigure enregistrée dans {FIG}")
```